The Shed, explained

A file and folder browser on the starter a developer test brief came with, built the way I build my own project: shape and tests first, every decision written down, and a live site.

DEVELOPER TEST PROJECT | STEVEN STOUT | 16 OCTOBER 2026

LIVE theshed.stevenstout.biz
CODE github.com/SteveStout/TheYard, samples/theshed
FULL SIZE theyard.stevenstout.biz
RESUME AND ABOUT on the site, one click each

What it is

One configurable home directory, one API, one page. The address bar is the whole state.

An ASP.NET Core 10 API over a home directory set by configuration, and a page in TypeScript with no framework. It browses, searches by name or wildcard, uploads with a progress bar, downloads, makes folders, moves, copies and deletes, inside a native dialog with one trigger. Every failure is a sentence the page shows. Fourteen decision records are served from the running app, each with the code it decided about read from the build as you read it.

QUICK START

```
dotnet run                # localhost:5120, browsing sample-home/
dotnet test               # 143 xUnit tests, warnings as errors
npm install && npm test   # 21 node tests over the compiled modules
```

OR SKIP THE BUILD, THREE LINKS

- **theshed.stevenstout.biz/?path=reports/2026**: the dialog, open in a folder.
- **/?q=*.csv&sort=size&dir=desc**: a search, sorted, as a link.
- **/?view=docs&doc=adr-003-the-line-a-path-cannot-cross**: the security record, its code live. Read ADR-003, 005 and 009 first; the README is the map.

By the numbers

164	< 1 ms	44 ms
tests, green before every commit	to browse 100 files, a test prints it	to search 10,000 files; capped, 8 ms
0	**14**	**$0**
warnings, analyzers at their strictest	records served live with their code	added hosting, on a plan I already pay for

Over 10,000 files on the build machine (ADR-008) *lower is better*

Browse, 100 files	< 1 ms
Search, capped at 200	8 ms
Search, whole tree	44 ms

The brief, and where each item is

The brief asked for	Where it is, under samples/theshed (its code: pages 3 to 5)
A. Builds in VS, Rider, VS Code, CLI	TestProject.sln, the starter's file kept by name, on .NET 10; no package.
B. JSON API: browse and search	Controllers/FilesController.cs; every failure an RFC 9457 document from Controllers/ProblemResponseHandler.cs.
C. Easy folder configuration	Files:Home or FILES__HOME; blank shows sample-home. Every path that would leave it is refused before the disk is touched.
D. Deep-linkable state in the URL	src/lib/urlState.ts parses and writes six keys; src/navigation.ts has the one navigate().
E. Pure TypeScript	src/ compiled by tsc alone to wwwroot/js; no framework, no bundler.
F. Upload and download	src/lib/api.ts uploads with a bar per file; downloads stream with ranges.
G. Counts and sizes per view	Domain/ViewTotals.cs, on every reply from Application/FileBrowser.cs, plus took_ms.
H. Bonus: dialog, writes, performance	A native dialog in wwwroot/index.html; delete, move, copy in Controllers/FilesController.cs; timing, piece 7.

Six choices, and why

1 The starter, kept
Its names, its controller shape, its page; five settings in the project file (ADR-001).

2 Dependencies inward
Onion architecture in one project, seven folders. A test fails outward links (ADR-013).

3 One pure security class
Innermost ring: three string refusals, then containment, tested with no disk (ADR-003).

4 The address is the state
Nothing is kept elsewhere. A search's first keystroke pushes; the rest replace (ADR-005).

5 Measured, not claimed
Search is lazy, stops at a cap and says so; a test prints the timings each run (ADR-008).

6 Rules beside their tests
A table of every standing rule and the test that holds it; a test reads the table (ADR-009).

How it was built and shipped

- **Green before every commit.** 164 tests and warnings as errors, then a scripted headless Chrome pass of the running page before each ship (deep links, search, Back, upload, delete, Escape), not yet in the repository, with zero console errors.
- **Live at no added cost.** A third web app on the App Service plan my project runs on, its own free managed certificate, the image in the same registry, rolled by a workflow whose identity holds Website Contributor on that one site and nothing wider. The Bicep and the workflow are in the folder. The live sample is open to every visitor, uploads and deletes included; a restart or a redeploy puts its files back.
- **Built with AI, and written down (ADR-011).** I decided the shape and the test list first, Claude drafted against them, the build ran on my machine, and I read everything before it shipped. My first push had a folder-name case collision Windows hides and Linux does not; the container found it, and the link test now checks case exactly.

Taking it further: my larger project is the worked example

TheYard (start here) is the same practice at full size: .NET 10 and React, Azure SQL and Cosmos DB, 90 records, about 2,100 test runs per gate, two sites on one plan. Each step below is something the Yard already does.

Next on the Shed	How the Yard already does it
A committed browser test	144 Playwright specs in the gate; axe holds eleven views to WCAG 2.1 AA. TheYard: the accessibility check
Folder sizes, a browse cap, paging	Both next. The Yard's API pages 100,000 vehicles behind Load more. TheYard: the vehicles API
Logs that outlive the container	Every request and error kept in Cosmos DB for three years (ADR-071, 073). TheYard: the kept logs
The path guard as a Files card	The Admin tab is a product: health, machines, timing, logs, each a card (ADR-080). TheYard: the Admin tab

Every requirement and bonus, done

The brief's words, where each is under samples/theshed, what proves it, and its code page. Tests: a fresh clone, 4 October 2026.

7 of 7 requirements done | **4 of 4** other rules done | **4 of 4** bonuses done

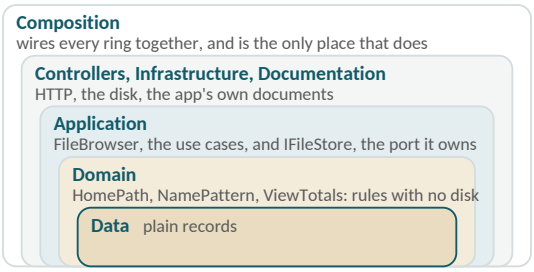
Requirements, in the brief's words		Where it is	What proves it	Page
Ensure your solution builds in Visual Studio (any version 2022 or newer is acceptable), Rider, VS Code, or command line SDK tools	Done	TestProject.sln and .csproj, the starter's names kept, on .NET 10.	This folder alone, from a fresh clone: 0 warnings, 0 errors; 143 of 143 tests pass. Visual Studio, Rider and VS Code build the same .sln; Visual Studio needs 2026 or newer for .NET 10 (ADR-001).	3
Web API that allows Browse and Search Files & Folders and returns JSON	Done	Controllers/FilesController.cs over Application/FileBrowser.cs.	FilesApiTests: browse home is snake case with totals and timing; search finds by glob and reports truncation. Live: a search.	3, 9
Deep linkable URL pattern	Done	src/lib/urlState.ts and src/navigation.ts.	urlState.test.js: parse and serialize round-trip. Live: the notes folder, as a link.	4, 10
SPA (Single Page App using JavaScript)	Done	wwwroot/index.html loads js/main.js, built from src/main.ts.	FrontEndRulesTests (ProjectRulesTests.cs): every source module has its compiled module beside the page. Live: opening a folder changes the address, never the page.	4, 9
Upload/download files from the browser	Done	src/lib/api.ts; FilesController Upload and Download.	FilesApiTests: upload then download round trips the bytes; upload past the limit is 413.	4
Show file and folder counts and sizes for the current view	Done	Domain/ViewTotals.cs, on every listing.	FilesApiTests: browse home is snake case with totals and timing. Live: the reports folder's totals.	5
Build the UI using vanilla JavaScript or TypeScript (without React, Angular, or other UI library)	Done	src/, compiled by tsc; one dev dependency, TypeScript.	FrontEndRulesTests (ProjectRulesTests.cs): no source uses innerHTML and only the compiler is a dependency.	4

The brief's other rules		Where it is	What proves it	Page
A server side home directory should be configurable via variable.	Done	Files:Home in appsettings.json, or FILES__HOME.	HomeForTests (FilesRegistrationTests.cs): empty configuration means the sample home; an absolute home is used as given. FilesApiTests: the home is the configured folder.	3
the state of the UI should be kept in the URL	Done	src/lib/urlState.ts: six keys, nothing kept elsewhere (ADR-005).	urlState.test.js: a bare address is closed and at the defaults. Live: Back reopens the dialog where it was.	4, 10
All of the UI work should be done client side via JavaScript that renders HTML - do not render HTML server side.	Done	src/ui builds every element; src/lib/markdown.ts renders the Docs tab.	A fresh clone has no Razor and no views. Live: every /api route answers JSON or markdown, never HTML.	4, 9
a primitive end to end flow that runs without errors	Done	Page, API, rules and disk, in one project.	FilesApiTests run the real app in memory; 143 xUnit and 21 node tests pass. Live: the links on page 1 open in headless Chrome before each ship, with 0 console errors.	8

Bonus, in the brief's words		Where it is	What proves it	Page
Any cool stuff you want to show :)	Done	The Docs tab: 14 records, their code read from the build. OnionTests. The live site, at no added cost.	DocumentationTests: every live fence resolves to code from this build. OnionTests: nine checks over the compiled app. Live: the onion record.	6, 7
Entire component contained in a dialog widget, with a trigger element (button, etc)	Done	A native dialog and its trigger button, in wwwroot/index.html (ADR-007).	Live, in headless Chrome before each ship: the button opens it, Escape closes it, Back reopens it. The dialog record.	5
Delete, move, copy files and folders	Done	FilesController Delete, Move, Copy; the yes is asked in the row.	FilesApiTests: folder create, delete, move, copy. FileTransferTests: move renames and relocates; copy leaves the source in place; a folder cannot go into itself.	5
Performance - performance is highly valued	Done	Application/FileBrowser.cs walks lazily, stops at a cap and says so.	PerformanceTests times 10,000 files on every run: under 1 ms to browse, 8 ms capped, 44 ms whole tree (ADR-008). FileBrowserTests: search stops at the limit and says so.	9

Onion and SOLID: how the code holds them

Each folder is a ring. A ring may use the rings inside it, never one outside. OnionTests reads the compiled app and fails the build when one reaches out (ADR-013).



SOLID, in plain words	Where it shows in this code
Single responsibility: one reason to change	FileBrowser holds the rules, the controllers hold HTTP, PhysicalFileStore holds the disk.
Open/closed: add a class, do not edit one	Cloud storage would be one new IFileStore class and one line in Composition; nothing else changes.
Liskov substitution: any version fits	The tests swap in a store in memory, and every rule runs the same.
Interface segregation: nothing unused	IFileStore has one user, FileBrowser, and it needs every method.
Dependency inversion: the inside owns the plug	Application declares IFileStore, Infrastructure fills it in, Composition plugs it in.

The brief, with the code

Read from the repository at 1.0.0.41. Samples keep their comments; a cut inside one is marked …; comment lines are rewrapped to the page width.

A. Builds in VS, Rider, VS Code, CLI

It opens and builds in Visual Studio, Rider, VS Code or the command line on .NET 10, with no extra packages, and the starter's solution and project file keep their names. Five settings were added: warnings stop the build, the code checkers run in the build (the sealed rule sits in .editorconfig), doc comments are generated, culture is invariant, and the documents, the sample home and the sources travel with a publish. Why it matters: any developer opens it in their own tool, and problems surface on their machine before a reviewer sees them. TheYard: The rules a change has to pass

samples/theshed/TestProject.csproj

```
<!-- The starter's project file, kept as the brief sent it (nullable, implicit usings) on net10.0, the
     same framework as TheYard (ADR-001), with five settings: warnings are errors, the code checkers
     run in the build (the sealed rule sits in .editorconfig), doc comments are generated, culture is
     invariant, and the documents, the sample home and the sources travel with a publish so the
     container serves them (ADR-001). -->
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
  <AnalysisLevel>latest-recommended</AnalysisLevel>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
  <InvariantGlobalization>true</InvariantGlobalization>
</PropertyGroup>
```

B. JSON API: browse and search

The page talks to the server through a small JSON API, one line per route in Controllers/FilesController.cs. When something goes wrong, the answer always has the same shape: a standard error document (RFC 9457) from Controllers/ProblemResponseHandler.cs (piece 6). Why it matters: a developer can call it from any tool without reading the code, and every team that uses it handles errors one way. TheYard: The API describes itself

samples/theshed/Controllers/FilesController.cs, Browse and Search

```
/// <summary>Lists one folder's contents. GET /api/files?path=docs</summary>
/// <param name="path">A folder, relative to home; empty means home.</param>
[HttpGet]
public Listing Browse([FromQuery] string? path) => browser.Browse(path);

/// <summary>
/// Finds everything under a folder whose name matches the query. GET /api/files/search?path=&amp;q=*.md&amp;limit=200
/// </summary>
/// <param name="path">The folder to search under; empty means home.</param>
/// <param name="q">A substring, or a glob pattern using * and ?.</param>
/// <param name="limit">The maximum number of matches to return.</param>
[HttpGet("search")]
public SearchResult Search([FromQuery] string? path, [FromQuery] string? q, [FromQuery] int? limit) => browser.Search(path, q, limit);
```

C. Easy folder configuration

The app shows one folder. Pick it in appsettings.json on the line Files:Home, or on a server with the setting FILES__HOME, which wins. Pick nothing and it shows the practice folder that ships with it, sample-home. Whichever folder it is, every path that would leave it is refused before the disk is touched: Domain/HomePath.cs (piece 4) is the guard. Why it matters: a developer clones and runs with no setup, and an organization ships one build to every server, each pointed at its own folder. TheYard: The code is public and the secrets are not

samples/theshed/appsettings.json and Composition/FilesRegistration.cs, HomeFor

```
"Files": {
  "Home": "",
  "MaxUploadBytes": 104857600,
  "SearchLimit": 200,
  "SearchCeiling": 1000
}

/// <summary>The practice folder that ships with the project, shown when no folder is set.</summary>
private const string PracticeFolder = "sample-home";

/// <summary>
/// Picks the folder the app shows and hands it to HomePath, the guard that keeps every request inside it. The folder comes from Files:Home in
/// appsettings.json, or from the FILES__HOME environment variable, which wins. Nothing set means the sample-home practice folder.
/// </summary>
/// <param name="options">The Files section of configuration.</param>
/// <param name="contentRoot">The folder the project runs from.</param>
public static HomePath HomeFor(FilesOptions options, string contentRoot)
{
    // Which folder was chosen? Nothing chosen means the practice folder.
    string chosen = string.IsNullOrWhiteSpace(options.Home) ? PracticeFolder : options.Home;

    // A full path (C:\files or /srv/files) is used as written. A short one (my-files) is taken from the folder the project runs from.
    string folder = Path.IsPathRooted(chosen) ? chosen : Path.Combine(contentRoot, chosen);

    // Make the folder if it is not there yet, so a fresh copy runs with no setup.
    Directory.CreateDirectory(folder);

    // From here on, HomePath is the only code that turns a request into a real path on disk, and it refuses any path that would leave this folder.
    return new HomePath(folder);
}
```

The brief, with the code, continued

Read from the repository at 1.0.0.41. Samples keep their comments; a cut inside one is marked ...; comment lines are rewrapped to the page width.

D. Deep-linkable state in the URL

Everything on screen (the folder, the search, the sort) is written in the address bar. src/lib/urlState.ts reads and writes it (piece 9), and src/navigation.ts changes it in one place (piece 10). A setting left at its default is not written, so links stay short. Why it matters: copy the address and anyone opens exactly what you see, so a bug report carries its own steps, and Back and Forward just work. TheYard: Every diagram opens on its own page

samples/theshed/src/lib/urlState.ts, DEFAULTS

```
/**
 *  The value of each key when the address does not set it: the file browser view, the top folder, no search, no document, sorted by name in ascending
 *  order. An address with none of these keys means the dialog is closed.
 */
export const DEFAULTS: Readonly<Omit<State, 'open'>> = Object.freeze({
  view: 'browse',
  path: '',
  q: '',
  sort: 'name',
  dir: 'asc',
  doc: '',
});
```

E. Pure TypeScript

Plain TypeScript, compiled by tsc alone: no framework, no bundler. src/lib holds the logic and src/ui draws the screen. Why it matters: a new developer reads it top to bottom, and there is no framework upgrade to schedule. TheYard: The React configuration, explained

samples/theshed/tsconfig.json and wwwroot/index.html

```
// TypeScript with nothing else (ADR-006): the sources in src/ compile to plain ES modules in wwwroot/js, which the page loads as they are. No
// bundler, no framework. The output is committed so `dotnet run` works with the .NET SDK alone; `npm run build` regenerates it.
"compilerOptions": {
  "target": "ES2022",
  "lib": ["ES2022", "DOM", "DOM.Iterable"],
  "module": "ES2022",
  "moduleResolution": "bundler",
  "rootDir": "src",
  "outDir": "wwwroot/js",
    ...
  },
  "include": ["src/**/*.ts"]
}
```

```
<script type="module" src="/js/main.js"></script>
```

F. Upload and download

Every upload shows its own progress bar (src/lib/api.ts uses XMLHttpRequest, because fetch cannot report upload progress). Downloads stream, and a broken one can pick up where it stopped. Why it matters: a user sees a big upload moving instead of a frozen screen, and a dropped download on a slow line resumes instead of starting over. TheYard: Cache headers

samples/theshed/src/lib/api.ts, upload and Controllers/FilesController.cs, Download

```
/**
 *  Uploads one file and reports progress while it goes. It uses XMLHttpRequest because fetch cannot report upload progress, and a large file needs a
 *  progress bar so the person can see it is still moving.
 * @param path the folder to upload into
 * @param file the file to send
 * @param overwrite true to replace a file with the same name; with false the server answers 409
 * @param onProgress called with the fraction sent so far, from 0 to 1
 */
export function upload(path: string, file: File, overwrite: boolean, onProgress: (fraction: number) => void): Promise<TransferResult> {
  return new Promise((resolve, reject) => {
    const form = new FormData();
    form.append('files', file, file.name);
    const request = new XMLHttpRequest();
    request.open('POST', `/api/files/upload?path=${encodeURIComponent(path)}&overwrite=${overwrite ? 'true' : 'false'}`);
    request.upload.addEventListener('progress', (event) => {
      if (event.lengthComputable) {
        onProgress(event.loaded / event.total);
      }
    });
    ...
```

```
/// <summary>
/// Sends a file's bytes as a download. Range requests are enabled and the file's last change time is sent with it, which is what a browser needs to
/// resume a paused or broken download where it stopped. The stream comes from the file store and is closed once the response is sent.
/// </summary>
/// <param name="path">A file, relative to home.</param>
[HttpGet("download")]
public IActionResult Download([FromQuery] string? path)
{
    (Stream content, FileEntry entry) = browser.Download(path);
    return new FileStreamResult(content, "application/octet-stream")
    {
        FileDownloadName = entry.Name,
        LastModified = DateTimeOffset.FromUnixTimeMilliseconds(entry.ModifiedMs),
        EnableRangeProcessing = true,
    };
}
```

The brief, with the code, continued

Read from the repository at 1.0.0.41. Samples keep their comments; a cut inside one is marked …; comment lines are rewrapped to the page width.

G. Counts and sizes per view

Every listing says how many folders and files it shows and how big they are, added up by Domain/ViewTotals.cs from exactly what is listed, plus took_ms, how long the server took. Why it matters: a user sees how much a view holds at a glance, and a developer sees the server's own timing on every reply, so a slow request is spotted without extra tools. TheYard: The listing that went stale while you looked at it

samples/theshed/Domain/ViewTotals.cs and Application/FileBrowser.cs, Browse

```
/// <summary>Counts the folders and files and adds up the files' sizes.</summary>
/// <param name="folders">The folders in the result.</param>
/// <param name="files">The files in the result.</param>
public static Totals Of(IReadOnlyList<FolderEntry> folders, IReadOnlyList<FileEntry> files)
{
    long bytes = 0;
    foreach (FileEntry file in files)
    {
        bytes += file.SizeBytes;
    }
    return new Totals(folders.Count, files.Count, bytes);
}

/// <summary>
/// Lists one folder's direct contents with totals and timing. Folders come first, then files, and each group is sorted by name ignoring case.
/// </summary>
/// <param name="path">The folder, relative to home; "" means home itself.</param>
public Listing Browse(string? path)
{
    long started = Stopwatch.GetTimestamp();
    string absolute = Folder(path);
    (List<FolderEntry> folders, List<FileEntry> files) = SplitByKind(store.Children(absolute));
    folders.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Name, b.Name));
    files.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Name, b.Name));
    string relative = home.Relative(absolute);
    return new Listing(relative, HomePath.ParentOf(relative), folders, files, ViewTotals.Of(folders, files), Elapsed(started));
}
```

H. Bonus: dialog, delete, move, copy

The file browser opens in the page's own dialog box (wwwroot/index.html), so the keyboard and Escape work with no extra code. Delete, move and copy are one route each in Controllers/FilesController.cs, and the page asks for a yes in the row itself, never a pop-up. The performance bonus is piece 7, with the test that times it. Why it matters: it works from the keyboard from day one, and a confirmation sits next to the thing it changes. TheYard: Keyboard and screen reader

samples/theshed/wwwroot/index.html and Controllers/FilesController.cs, Delete, Move and Copy

```
<dialog class="shed" aria-labelledby="shed-title">
  <div class="panel">
    <div class="dialog-head">
      <h2 id="shed-title">The Shed</h2>
      <div class="seg" id="tabs" role="group" aria-label="View">
        <button type="button" data-view="browse" aria-pressed="true">Files</button>
        <button type="button" data-view="docs" aria-pressed="false">Docs</button>
      </div>
      <span class="spacer"></span>
      <button type="button" id="close-shed" class="close" aria-label="Close the file browser">&times;</button>
    </div>
    <section id="browser" aria-label="Files"></section>
    <section id="docs" class="docs" aria-label="Documents" hidden></section>
  </div>
</dialog>
```

```
/// <summary>
/// Deletes a file, or a folder and everything in it, and answers 204. DELETE /api/files?path=docs/old.txt
/// </summary>
/// <param name="path">The file or folder to delete.</param>
[HttpDelete]
public IActionResult Delete([FromQuery] string? path)
{
    browser.Delete(path);
    return NoContent();
}

/// <summary>
/// Moves a file or folder and returns it at its new location. POST /api/files/move with {"from": "a/b.txt", "to": "c/b.txt"}
/// </summary>
/// <param name="request">The source and destination, both relative to home.</param>
[HttpPost("move")]
public object Move([FromBody] MoveRequest request) => browser.Move(request.From, request.To);

/// <summary>
/// Copies a file or folder and returns the new copy. POST /api/files/copy with {"from": "a/b.txt", "to": "c/b.txt"}
/// </summary>
/// <param name="request">The source and destination, both relative to home.</param>
[HttpPost("copy")]
public object Copy([FromBody] MoveRequest request) => browser.Copy(request.From, request.To);
```

The code, in ten pieces

Read from the repository at 1.0.0.41. Samples keep their comments; a cut inside one is marked ...; comment lines are rewrapped to the page width.

1. Program.cs is a table of contents

Program.cs reads like a table of contents: each line names one part of the app, and the file beside it holds the details. There are no rules in it, only the order things start in. src/main.ts does the same for the page. Why it matters: a new developer learns what the app is made of in one screen, and a reviewer knows exactly where to look for each part. Docs tab record: One project, seven folders, dependencies inward. TheYard: Program.cs, explained for a new developer

samples/theshed/Program.cs

```
/// <summary>
/// The entry point. It builds the web host and starts it. Each step is a single call, and the comment beside the call names the file that holds the
/// details. The setup lives in those files so that this class stays a short, readable list of what the app is made of.
/// </summary>
public sealed class Program
{
    /// <summary>Builds the web app from its parts, in order, and runs it until shutdown.</summary>
    /// <param name="args">Command line arguments, passed to the host builder.</param>
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        builder.AddTheShedFiles();                  // Composition/FilesRegistration.cs
        builder.AddTheShedDocumentationAndVersion();  // Composition/DocumentationAndVersionRegistration.cs
        builder.AddTheShedApi();                    // Composition/ApiRegistration.cs

        var app = builder.Build();
        app.UseTheShedRequestPipeline();            // Composition/RequestPipeline.cs
        app.MapControllers();                       // Controllers/: Files, DocumentationAndVersion, Health

        app.Run();
    }
}
```

2. The file browser, registered

This file sets up the file browser: the folder to show (item C), the rules and the upload limit, so an oversized file gets a message that names it. The rules reach the disk only through the IFileStore interface; the sample keeps files on the server's own disk. Why it matters: moving to Azure Blob Storage is one new class and one line here, and nothing else changes. Docs tab records: One project, seven folders, dependencies inward; The line a path cannot cross. TheYard: The composition root, split by job

samples/theshed/Composition/FilesRegistration.cs, AddTheShedFiles

```
/// <summary>
/// Registers the file browser's services: the home directory and its limits, the file store interface with the class that reads and writes real
/// files, and the FileBrowser that holds the rules. It also sets the server's upload size limits to match the configured maximum.
/// </summary>
public static class FilesRegistration
{
    /// <summary>
    /// Adds the home directory, the file store, the FileBrowser and the upload size limits to the host.
    /// </summary>
    /// <param name="builder">The host being built.</param>
    public static void AddTheShedFiles(this WebApplicationBuilder builder)
    {
        // Reads the "Files" section of appsettings.json into FilesOptions. An environment variable named FILES__ plus the setting (FILES__HOME, for
        // example) replaces the value from the file, which is how a server chooses its folder without editing the code. The options and the HomePath
        // built from them are registered once, so every class sees the same folder. The limits are checked once as the app starts (IsUsable), so a
        // setting that cannot work stops the app with one message instead of turning a later search or upload into a 500.
        builder.Services.AddOptions<FilesOptions>()
            .Bind(builder.Configuration.GetSection(FilesOptions.Section))
            .Validate(IsUsable, SettingsProblem)
            .ValidateOnStart();
        builder.Services.AddSingleton(provider => provider.GetRequiredService<IOptions<FilesOptions>>().Value);
        builder.Services.AddSingleton(provider => HomeFor(provider.GetRequiredService<FilesOptions>(), builder.Environment.ContentRootPath));

        // IFileStore is the interface the rules use to touch the disk, and PhysicalFileStore is the class that reads and writes real files.
        // FileBrowser holds the rules and depends only on the interface, so tests can swap in a different store.
        builder.Services.AddSingleton<IFileStore, PhysicalFileStore>();
        builder.Services.AddSingleton<FileBrowser>();

        // FileBrowser.UploadAsync checks the exact per-file limit and names the file in its error. The web server and the form parser get that limit
        // plus one megabyte, to allow for the multipart form's own headers and boundaries. A slightly large request therefore still reaches the
        // per-file check and gets a clear message, while a far too large request is cut off by the web server with a 413 once it passes that limit.
        long maxUpload = builder.Configuration.GetSection(FilesOptions.Section).GetValue("MaxUploadBytes", new FilesOptions().MaxUploadBytes);
        long slack = 1024 * 1024;
        builder.WebHost.ConfigureKestrel(kestrel => kestrel.Limits.MaxRequestBodySize = maxUpload + slack);
        builder.Services.Configure<FormOptions>(form => form.MultipartBodyLengthLimit = maxUpload + slack);
    }
    ...
}
```

3. The documents and the version, registered

This file sets up the version number and the documents. The version is the top line of docs/CHANGELOG.md, the commit comes from the deploy, and the documents come from the docs folder. Why it matters: the version on screen can never disagree with the changelog, so support and developers always know exactly which build they are looking at. Docs tab record: Documents served by the app, with live code. TheYard: Live code samples

samples/theshed/Composition/DocumentationAndVersionRegistration.cs

```
/// <summary>
/// Registers the two services behind the Docs tab and the version in the page footer. This file does not set the version or write any documentation.
/// It only makes the readers available to the controllers:
/// <list type="bullet">
/// <item>The version number is set by the newest entry at the top of docs/CHANGELOG.md.
/// Adding a line there is how a new version is set.</item>
/// <item>The commit is set by the deploy, which puts it in the SHED_COMMIT environment
/// variable of the container. A local run reads it from the .git folder instead.</item>
/// <item>The documentation is the markdown files in the docs folder (the decision records
/// and the guides) plus README.md. DocumentationAndVersionController serves them to the Docs tab.</item>
/// </list>
/// </summary>
public static class DocumentationAndVersionRegistration
{
    /// <summary>
    /// Adds DocumentationCatalog, which lists the documents in sidebar order and finds one by name, reading them from disk on every request so an
    /// edited file shows on the next page load. Adds VersionInfo, which VersionReader reads once at startup from the changelog and the commit sources
    /// above. Both are singletons rooted at the content root.
    /// </summary>
    /// <param name="builder">The host being built.</param>
    public static void AddTheShedDocumentationAndVersion(this WebApplicationBuilder builder)
    {
        builder.Services.AddSingleton(new DocumentationCatalog(builder.Environment.ContentRootPath));
        builder.Services.AddSingleton(VersionReader.Read(builder.Environment.ContentRootPath));
    }
}
```

4. The line a path cannot cross

This is the guard. Every path a request asks for passes through here, and anything that tries to step outside the chosen folder is refused before the disk is touched. It sits in the innermost layer (onion architecture) with no disk code, so it is tested on its own. Why it matters: the most important security rule in the app is one small class a reviewer can read in a minute. Docs tab record: The line a path cannot cross. TheYard: Security

samples/theshed/Domain/HomePath.cs, Resolve

```
/// <summary>
/// Turns a path from a request into an absolute path under the root, or throws <see cref="PathRefusedException"/>. It first refuses three things by
/// looking at the string alone: a rooted path (a drive letter or a leading slash), a "." or ".." segment anywhere (a segment is one name between
/// slashes), and a segment containing a character the operating system does not allow in a name. It then joins the segments onto the root, lets the
/// operating system work out the final full path, and checks that path still lies inside the root. That final check is the safety net for any trick
/// the first three checks do not anticipate.
/// </summary>
/// <param name="relative">The path the request sent; null or "" means home itself.</param>
public string Resolve(string? relative)
{
    string trimmed = (relative ?? string.Empty).Trim();
    if (trimmed.Length == 0)
    {
        return Root;
    }
    if (System.IO.Path.IsPathRooted(trimmed) || trimmed.StartsWith('/') || trimmed.StartsWith('\\'))
    {
        throw new PathRefusedException("A path must be relative to the home directory.");
    }
    string[] segments = trimmed.Split(Separators, StringSplitOptions.RemoveEmptyEntries);
    foreach (string segment in segments)
    {
        if (segment is "." or "..")
        {
            throw new PathRefusedException("A path cannot contain '.' or '..' segments.");
        }
        if (segment.IndexOfAny(System.IO.Path.GetInvalidFileNameChars()) >= 0)
        {
            throw new PathRefusedException($"'{segment}' is not a valid name.");
        }
    }
    // Windows drops trailing dots and spaces from a name, so "..." or "x " can come back as the root with a separator on the end. Trimming the
    // separators hands back the root in the one spelling every caller compares against, so home is always recognised as home.
    string joined = System.IO.Path.GetFullPath(System.IO.Path.Combine(Root, System.IO.Path.Combine(segments))).TrimEnd(Separators);
    if (!IsInside(joined))
    {
        throw new PathRefusedException("A path must stay inside the home directory.");
    }
    return IsRoot(joined) ? Root : joined;
}
```

5. The tests that hold it

Each way out has its own test, including the one most guards miss: /home/files-old starts like /home/files but is not inside it. Why it matters: the build fails before anyone weakens the guard. TheYard: The tests, explained for a new developer

samples/theshed/tests/TestProject.Tests/HomePathTests.cs

```
/// <summary>
/// Checks HomePath, the guard that keeps every requested path inside the home folder, with plain strings and no disk. Each way of escaping it (dot
/// segments, rooted paths, bad characters, shared prefixes) has its own case. (more in docs/ADR-003-the-line-a-path-cannot-cross.md)
/// </summary>
public sealed class HomePathTests
{
    ...
    [Theory]
    [InlineData("../secrets")]
    [InlineData("docs/../../secrets")]
    [InlineData("docs/./notes")]
    [InlineData("..")]
    public void A_dot_or_dot_dot_segment_is_refused(string path)
    {
        var refused = Assert.Throws<PathRefusedException>(() => _home.Resolve(path));
        Assert.Contains("'.' or '..'", refused.Message, StringComparison.Ordinal);
    }
    ...
    [Fact]
    public void The_root_with_a_prefix_in_common_is_outside()
    {
        // /home/files-old starts with the same characters as /home/files but is a different folder. A plain string prefix check would wrongly call it
        // inside.
        Assert.False(_home.IsInside(_home.Root + "-old"));
        Assert.False(_home.IsInside(Path.Combine(_home.Root + "-old", "x")));
    }
    ...
}
```

6. Every failure is a sentence

One handler turns every refusal into the right status code and a standard error document (RFC 9457) with a trace id. Why it matters: every error reads the same way for the user and the developer, and the trace id points a developer to that exact request. Docs tab record: The wire. TheYard: Error handling, one shape everywhere

samples/theshed/Controllers/ProblemResponseHandler.cs

```
/// <summary>
/// Turns a refused request into an error reply in RFC 9457, the web standard for errors: a status, a short title, the reason in one sentence and a
/// trace id. It is the one place errors get their shape, so no controller needs a try/catch.
/// </summary>
/// <remarks>
/// The app's own refusals carry their status; a path that leaves home is a 400; a request the web server cut off keeps its status (413 past the size
/// limit); the form reader's refusal is a 413 when it names a limit, else a 400; a file the disk will not let us touch (locked, read-only, gone, a
/// full disk) is a 403 or a 409 naming no path. Anything else is a real bug, a 500.
/// </remarks>
public sealed class ProblemResponseHandler(IProblemDetailsService problems) : IExceptionHandler
{
    /// <summary>
    /// Called by ASP.NET Core when a request throws. Returns true when it wrote the error reply, or false to pass the exception on to the next
    /// handler.
    /// </summary>
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext, Exception exception, CancellationToken cancellationToken)
    {
        // 1. Pick the status and title by the kind of exception, most specific first: a BadHttpRequestException is an IOException. A status of 0
        // means none of these.
        (int status, string title) = exception switch
        {
            ApiRefusalException problem => (problem.Status, problem.Title),
            PathRefusedException => (StatusCodes.Status400BadRequest, "The path was refused"),
            BadHttpRequestException bad => (bad.StatusCode, "The request was refused"),
            InvalidDataException invalid when NamesALimit(invalid) => (StatusCodes.Status413PayloadTooLarge, "The upload is too large"),
            InvalidDataException => (StatusCodes.Status400BadRequest, "The upload could not be read"),
            UnauthorizedAccessException => (StatusCodes.Status403Forbidden, "The filesystem refused"),
            IOException => (StatusCodes.Status409Conflict, "The filesystem refused"),
            _ => (0, string.Empty),
        };

        // 2. Not one we know: pass it on, so a real bug still shows up as a 500.
        if (status == 0)
        {
            return false;
        }

        httpContext.Response.StatusCode = status;

        // 3. Write the reply: the reason from ReasonFor, and the trace id the logs carry, so a failure a user reports can be found there.
        return await problems.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = httpContext,
            ProblemDetails = new ProblemDetails
            {
                Status = status,
                Title = title,
                Detail = ReasonFor(exception),
                Extensions = { ["traceId"] = Activity.Current?.Id ?? httpContext.TraceIdentifier },
            },
        });
    }
    ...
}
```

7. A search that stops

The search walks the folders one entry at a time and stops as soon as it has enough matches, and the reply says when it stopped early. Why it matters: a search over a huge drive still answers fast without tying up the server, and the user is told the list was cut short. Below it, PerformanceTests times 10,000 files on a real disk on every run. Docs tab record: Performance, measured. TheYard: The search index; TheYard: Same performance, proven

samples/theshed/Application/FileBrowser.cs, Search, and tests/TestProject.Tests/PerformanceTests.cs

```
/// <summary>
/// Finds every file and folder under a folder, at any depth, whose name matches the query, up to a maximum number of matches. The store hands back
/// entries one at a time as it walks, and <see cref="SearchMatches"/> stops the walk at the first match past the maximum, so a search that fills up
/// early never reads the rest of the tree (more in docs/ADR-008-performance-measured.md).
/// </summary>
/// <param name="path">The folder to search under, relative to home; "" means home.</param>
/// <param name="query">A plain substring, or a glob using * or ?; empty is refused.</param>
/// <param name="limit">
/// The most matches to return. Null uses the configured default, and any value is clamped between 1 and the configured ceiling. </param>
public SearchResult Search(string? path, string? query, int? limit)
{
    long started = Stopwatch.GetTimestamp();
    var pattern = new NamePattern(query);
    if (pattern.IsEmpty)
    {
        throw ApiRefusalException.Refused("A search needs something to look for.");
    }
    int cap = Math.Clamp(limit ?? options.SearchLimit, 1, options.SearchCeiling);
    string absolute = Folder(path);
    (List<StoreEntry> kept, bool truncated) = SearchMatches.Take(store.Descendants(absolute), pattern, cap);
    (List<FolderEntry> folders, List<FileEntry> files) = SplitByKind(kept);
    folders.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Path, b.Path));
    files.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Path, b.Path));
    return new SearchResult(pattern.Query, home.Relative(absolute), folders, files, ViewTotals.Of(folders, files), truncated, Elapsed(started));
}

// Browse once before timing to warm the OS directory cache, so the timings match a repeat visit rather than the first read of a cold disk.
browser.Browse("folder-000");

long browseMs = Time(() => browser.Browse("folder-042"), out int browsed);
long cappedMs = Time(() => browser.Search("", "*.txt", 200), out int capped);
long fullMs = Time(() => browser.Search("", "file-099-099", null), out int found);
long rootMs = Time(() => browser.Browse(""), out int folders);

output.WriteLine($"generated {Folders * FilesPerFolder:N0} files in {generated.ElapsedMilliseconds} ms");
output.WriteLine($"browse one folder of {browsed} files: {browseMs} ms");
output.WriteLine($"browse the root of {folders} folders: {rootMs} ms");
output.WriteLine($"search capped at {capped} matches: {cappedMs} ms");
output.WriteLine($"search the whole tree for one name ({found} match): {fullMs} ms");

Assert.Equal(FilesPerFolder, browsed);
Assert.Equal(200, capped);
Assert.Equal(1, found);
Assert.True(browseMs < 250, $"browsing one folder took {browseMs} ms");
Assert.True(cappedMs < 500, $"a capped search took {cappedMs} ms");
Assert.True(fullMs < 3000, $"a full-tree search took {fullMs} ms");
```

8. main.ts, where the page starts

The page's TypeScript starts here. index.html loads one script, js/main.js, which tsc compiles from src/main.ts. Like Program.cs, it is a table of contents: each line starts one part of the page, in order, with the file that holds the details beside it. Why it matters: a developer sees what the page is made of in one screen and goes straight to the file for the part they need. Docs tab records: One project, seven folders, dependencies inward; TypeScript, organised. TheYard: Projects

samples/theshed/src/main.ts

```
/**
 *  The entry point for the page. It reads like a table of contents: each line starts one part, in order, and the comment beside it names the file
 *  that holds the details. Program.cs does the same for the server.
 */

import { currentState, navigate, render, startNavigation } from './navigation.js';
import { connectControls } from './ui/controls.js';
import { findPageElements } from './ui/pageElements.js';
import { showVersion } from './ui/versionFooter.js';

const page = findPageElements();                // ui/pageElements.ts: the elements index.html must have
startNavigation(page);                          // navigation.ts: the address bar, the two views, Back and Forward
connectControls(page, navigate, currentState);  // ui/controls.ts: open, close, the tabs, view links, Escape
showVersion(page.footer, navigate);             // ui/versionFooter.ts: the build version in the footer
render();                                       // navigation.ts: draw what the address asks for
```

9. The address is the state

One function reads the address bar and one writes it. A value that makes no sense falls back to the default, and defaults are left out, so links stay short. Why it matters: an old or mistyped link still opens something sensible instead of an error, and a shared link stays readable. Docs tab record: State lives in the URL. TheYard: A record with no address

samples/theshed/src/lib/urlState.ts

```
/**
 *  Reads a query string into a State. Unknown keys are ignored. A value that is not in its allowed list falls back to the default. A path is cleaned
 *  to forward slashes with no leading or trailing slash, so two ways of writing the same folder give the same State.
 * @param search the query string from the address, with or without the leading "?"
 */
export function parse(search: string): State {
  const params = new URLSearchParams(search.startsWith('?') ? search.slice(1) : search);
  return {
    // Any known key opens the dialog, so a link as short as "?path=docs" works. An open dialog with every other value at its default is written as
    // "?view=browse" by serialize().
    open: KEYS.some((key) => params.has(key)),
    view: pick(params.get('view'), VIEWS, DEFAULTS.view),
    path: cleanPath(params.get('path')),
    q: (params.get('q') ?? '').trim(),
    sort: pick(params.get('sort'), SORTS, DEFAULTS.sort),
    dir: pick(params.get('dir'), DIRS, DEFAULTS.dir),
    doc: (params.get('doc') ?? '').trim().toLowerCase(),
  };
}

/**
 *  Writes a State as a query string. Values equal to their default are left out, so links stay as short as possible. A closed State gives "" (no
 *  query at all). An open State where every value is the default would also give nothing, so it writes "?view=browse" to keep the dialog open when
 *  the address is read back. @returns "" when closed, otherwise "?key=value&..."
 */
export function serialize(state: Partial<State>): string {
  const full: State = { ...DEFAULTS, open: true, ...state, path: cleanPath(state.path) };
  if (!full.open) {
    return '';
  }
  const params = new URLSearchParams();
  for (const key of KEYS) {
    if (full[key] !== DEFAULTS[key] && full[key] !== '') {
      params.set(key, full[key]);
    }
  }
  if (![...params.keys()].length) {
    params.set('view', full.view);
  }
  return `?${params.toString()}`;
}
```

10. The one way the state changes

Every change on screen goes through this one function: update the state, write the address, redraw. Closing the dialog is navigating back to the defaults, so Back reopens it where it was. Why it matters: one path for every change means one place to look for a bug, and the Back button always does what a user expects. Docs tab record: State lives in the URL. TheYard: UI architecture

samples/theshed/src/navigation.ts, navigate

```
/**
 *  Changes the page state. It merges the changed keys into the current state, writes the result into the address bar as a history entry, then
 *  redraws. Every state change in the app comes through here. Closing the dialog is also a navigation (to the default state), so pressing Back after
 *  closing reopens the dialog where it was.
 * @param partial the state keys that change; keys left out keep their current value
 * @param replaceEntry true to overwrite the current history entry instead of adding one
 *  (used while typing in the search box, so each keystroke is not its own Back step)
 */
export function navigate(partial: Partial<State>, replaceEntry = false): void {
  const next: State = { ...state, open: true, ...partial };
  // A closed state serializes to an empty string. pushState with "" would keep the old query string, so the page path with no query string is written
  // instead, which clears it.
  const url = serialize(next) || window.location.pathname;
  if (replaceEntry) {
    window.history.replaceState(null, '', url);
  } else {
    window.history.pushState(null, '', url);
  }
  // Parse the address again instead of keeping `next`. The address is the only source of truth, and parsing it also cleans up any value the
  // serializer left out or would have rejected.
  state = parse(window.location.search);
  render();
}
```